

Jai Adams

Jordan Shapiro

Edmond Nzivugira

COSC 465

Project Proposal

March 31, 2026

Project objective and motivation

Our project will aim to tackle the question: "How can distributed video sources be aligned in time when network conditions cause frames to arrive at different times?" In both broadcasting and other videography workflows, hardware solutions, like genlock, or post-production techniques are often utilized to sync up multiple feeds, but with the improvement of network speeds over the past decade, dynamically handling video synchronization over the network in real time is becoming increasingly more widespread. With an abundance of applications such as surveillance systems, television broadcasting, and autonomous vehicles, finding efficient solutions that explore the tradeoff between synchronization accuracy and latency is crucial.

Prior Artifacts

By implementing a network or software-based solutions, video can be synced regardless of camera type and video format, providing a more portable alternative to hardware solutions for video synchronization. However, there are some shortcomings faced by this approach. Sending video over the network limits performance by latency and network reliability. Additionally, there is the possibility for potential security or privacy threats associated with using a network for this task, which need to be mitigated [Cinedeck, 2023].

In Temporal Synchronization of Video Sequences in Theory and in Practice, Whitehead [2005] critiques the conventional "linear progressive" synchronization model, which assumes a constant and

uniform timeline across cameras. This approach aligns frames by shifting one sequence to match another but fails to account for factors such as frame jitter and variations in camera capture rates. As an alternative, Whitehead proposes a periodic or wave-based model, where frames are mapped according to their phase within a capture cycle determined by frame rate. Central to this approach is the concept of a Primary Synchronization Period, a mathematically derived interval where two cameras naturally realign, offering a more robust framework than guess-based alignment methods.

Since Whitehead [2005], there has been significant attention and development towards solving these challenges with synchronization. A major development with regards to this is the IEEE 1588 standard, as published by the IEEE [2024], which defines the Precision Time Protocol (PTP). PTP entails clock synchronization in the sub-microsecond range with minimal network bandwidth and local clock computing resources. The way this functions is that one device is elected as the grandmaster clock and keeps time in accordance with UTC. Then, the grandmaster sends sync messages to share its current time with other devices on the network. Receivers timestamp when sync messages arrive using hardware-assisted timestamping and each device calculates clock offset (difference from grandmaster clock) and path delay. Local clocks on the transmitting devices are then adjusted in accordance with the grandmaster clock.

Lastly, the resources on OpenCV below will prepare our team to carry out video capture and visualization as needed in order to complete our project.

- [OpenCV documentation](#)
- (Video) [OpenCV tutorial for beginners](#)
 - Details how to read and visualize images and videos using opencv
- (Video) [Reading video streams from cameras connected to the computer using OpenCV](#)
 - Details how to use cameras with OpenCV and visualize multiple video feeds

Project plan

We'll follow an experiment-driven approach to implement this project. First, we'll build a system that captures video streams from two sources (Either directly from webcams or pre-recorded video files). This system will be built using Python and the OpenCV library. Each captured frame will be timestamped by the system and be transmitted to the server (any central receiver we'll choose) using sockets. The receiver will display videos side by side without synchronization and measure the initial misalignment, which will be our analysis baseline.

At the receiver, we will implement a synchronization algorithm based on some of the protocols mentioned in the survey-related-artifacts section (specifically the one based on buffering and timestamp alignment). The algorithm temporarily stores the frames in a buffer and releases them in a coordinated way to reduce time differences between streams. We will manually simulate network conditions such as delay and jitter to evaluate the performance/accuracy of our synchronization algorithm.

Accomplishments by the first status report:

- A system for capturing, transmitting, and displaying multiple video streams
- A demonstration showing unsynchronized streams
- Initial measurements of timing differences between streams

Accomplishments by the second status report:

- A synchronization algorithm (using buffering and timestamps)
- One or two experiments with simulated network delay
- Preliminary results comparing synchronization accuracy and latency

Accomplishments by the end of the project:

- A multi-camera synchronization system
- Quantitative analysis of tradeoffs between latency and synchronization accuracy
- Side-by-side demonstration of streams before and after synchronization
- A written report and GitHub repository with all the code and necessary files used

We'll use GenAI to accelerate development and deepen analysis. Specifically, we'll use tools like Github Copilot to generate tests, debug our scripts, and design experiments. Depending on the early results and experiments, we might use GenAI to extend the project using FFmpeg for realistic streaming and encoding behavior. The reason for this option is that although OpenCV gives us a lot of flexibility in manually designing synchronization algorithms, FFmpeg is more realistic and has a built-in buffering mechanisms and supports both RTP (UDP-based) and HTTP streaming. And since FFmpeg is harder to learn compared to OpenCV, using GenAI to accelerate our understanding of the framework will be a necessity.

Artifacts to deliver

A written report will contain system design, experimental methodology, and findings. An accompanying GitHub repository will provide code of our multi-camera synchronization system utilized to measure the synchronization accuracy of various video inputs at different network latencies. This codebase will include instructions for reproducibility, as well as a following dataset to be analyzed in the final report, which will be our findings.

References:

Aleksandar Haber. 2023. Reading video streams from cameras connected to the computer using OpenCV. Video. (14 Dec 2023). Retrieved from <https://www.youtube.com/watch?v=RnQy7U5Eu94>

Anthony Whitehead, Robert Laganieri, Prosenjit Bose. 2005. Temporal Synchronization of Video Sequences in Theory and in Practice. IEEE Workshop on Motion and Video Computing (WACV/MOTION'05). 10.1109/ACVMOT.2005.114.

Cinedeck. 2023. Exploring the Capabilities of IP-based Remote Production in the Broadcast Industry. Retrieved March 30, 2026 from <https://cinedeck.com/cinedeck-blog/exploring-the-capabilities-of-ip-based-remote-production-in-the-broadcast-industry/>

Computer vision engineer. 2023. OpenCV Tutorial for Beginners. Video. (27 Feb 2023). Retrieved March 31 from <https://www.youtube.com/watch?v=eDIj5LuIL4A>

IEEE Instrumentation and Measurement Society. 2024. IEEE P1588. Retrieved from <https://standards.ieee.org/ieee/1588/11795/>

OpenCV. Basic Operations on Images. Retrieved March 31 from https://docs.opencv.org/4.x/d3/df2/tutorial_py_basic_ops.html